End-to-End Machine Learning

Project Guidelines

From Problem Formulation to Reproducible Results

IMPACT Lab

Department of Computer Science and Engineering
Indian Institute of Technology Patna

November 5, 2026

Purpose and Expected Outcome

Goal

Develop a **scientifically sound, reproducible and clearly documented** ML solution to a well-defined problem—not merely a high accuracy score.

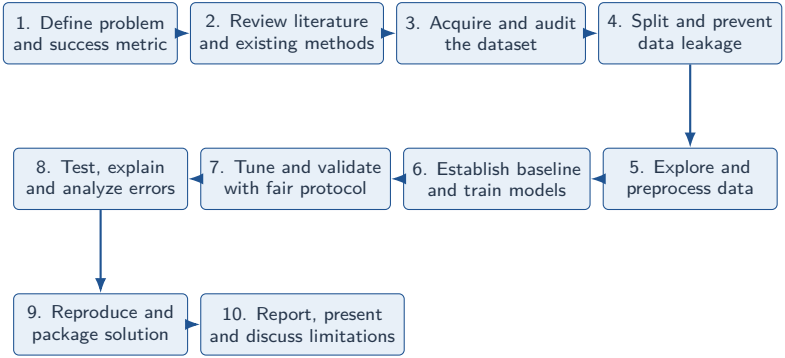
At project completion

- State a measurable problem and hypothesis.
- Justify dataset and preprocessing choices.
- Implement appropriate models and baselines.
- Evaluate fairly on unseen data.

Evidence to submit

- Working code and environment details.
- Results, plots and error analysis.
- Well-structured report and presentation.
- Limitations and reproducibility checklist.

Complete Project Roadmap



Core rule

Every methodological decision should be justified, recorded and verified.

Step 1: Define the Problem

- Identify the application and target users.
- Write a precise question and **scope**.
- Classify the task: regression, classification, clustering or forecasting.
- Define inputs, outputs, constraints and assumptions.
- Select success criteria *before* experiments.

Problem statement template

Given **[inputs]**, predict / discover **[target]** for **[population]** under **[constraints]**, achieving **[metric / threshold]**.

Deliverable

1-page problem definition and measurable objectives.

Step 2: Literature Survey and Gap Identification

- Review relevant recent papers, benchmarks and credible technical sources.
- Identify datasets, features, evaluation protocols and reported metrics.
- Compare existing methods **on comparable tasks and splits**.
- State the research or engineering gap; avoid unsupported novelty claims.
- Cite original datasets, algorithms, tools and borrowed ideas.

Suggested comparison table

Study	Dataset	Method	Evaluation	Gap / limitation
Paper A	Data A	Model A	Metric + split	Limitation A
Paper B	Data B	Model B	Metric + split	Limitation B

Step 3: Data Collection and Dataset Audit

Document the source

- Origin, license, citation and collection period.
- Unit of observation and target definition.
- Feature descriptions, units and data types.
- Data size, class balance and grouping variables.

Perform quality checks

- Missing values and duplicate records.
- Impossible values, anomalies and outliers.
- Label quality, selection bias, imbalance.
- Privacy, permissions and possible target proxies.

Deliverable

Dataset card + data dictionary + audit summary with counts and visualizations.

Step 4: Correct Splitting and Leakage Prevention

Freeze the evaluation protocol before training

Training	Fit preprocessing, features and model parameters.
Validation / CV	Compare algorithms and tune hyperparameters.
Held-out test	Final unbiased evaluation after selection; do not tune on it.

- Use stratification for imbalanced classification, when appropriate.
- Use group-wise splits for repeated subjects / devices / locations.
- Use chronological splits for forecasting; never shuffle future into past.
- Fit imputation, scaling, encoding, feature selection **inside training folds only**.

Stop sign

No data augmentation, oversampling, normalization or feature selection using held-out test information.

Step 5: Exploratory Data Analysis (EDA)

- Show sample counts, missingness and summary statistics.
- Plot target distribution and feature distributions.
- Examine correlations, drift and potential confounders.
- Investigate imbalance and subgroup differences.
- Record insights that guide later choices.

Minimum EDA evidence

- Target histogram / class-count plot.
- Missing-value summary.
- Feature distributions and relevant relations.
- Short interpretation under each figure.

Reminder

Use training data to make data-dependent modeling decisions.

Step 6: Preprocessing and Feature Engineering

- Select methods appropriate for feature types: numeric, categorical, text, time series or images.
- Handle missing values, categories and scaling in a **reusable pipeline**.
- Transform targets only when justified and reverse transforms for reporting.
- Engineer features based on domain knowledge, not knowledge of test labels.
- For imbalance, consider class weights or resampling **within training folds**.

Correct pattern

Raw data $\rightarrow$ split $\rightarrow$ fit transformer on training fold $\rightarrow$ transform validation/test with fitted transformer

Step 7: Establish a Credible Baseline

Start simple

- Regression: mean/median predictor; linear regression.
- Classification: majority-class; logistic regression.
- Clustering: meaningful simple grouping or reference partition.
- Forecasting: persistence or seasonal naive.

Then compare fairly

- Select several appropriate conventional ML models.
- Use identical splits and metrics.
- Record training time and model complexity when relevant.
- Explain *why* each model is included.

Key question

Does the proposed method materially outperform the simple baseline, and at what cost?

Step 8: Train and Tune the Models

- Fix seeds where feasible; record software versions and hardware.
- Use k-fold CV / grouped CV / rolling-origin validation as appropriate.
- Tune with predefined search spaces (grid, random or Bayesian search).
- Keep feature processing **inside each validation fold**.
- Avoid choosing hyperparameters from repeated test-set inspection.

Experiment log: essential columns

Run ID	Split / seed	Model	Hyperparameters	CV metric	Runtime
E01	Fold set A	Baseline	Default	Value	Time
E02	Fold set A	Model 2	Search config	Value	Time

Step 9: Choose Appropriate Evaluation Metrics

Task	Useful metrics	Checks to include
Regression	MAE, RMSE, R^2	Residuals, calibration across ranges, outliers
Classification	Precision, recall, F1, ROC-AUC / PR-AUC	Confusion matrix, per-class scores, threshold
Clustering	Silhouette, Davies–Bouldin; ARI if labels exist	Stability, interpretability, cluster sizes
Forecasting	MAE, RMSE, sMAPE (if suitable)	Horizon-wise error, chronological backtest

Important

Use metrics aligned to the goal. Report variance / confidence intervals when feasible, not just the best run.

Step 10: Final Testing and Fair Comparison

- Select the final pipeline using **training and validation only**.
- Refit on the development data, if justified by the protocol.
- Evaluate the untouched test set **once** for final reporting.
- Compare all candidates under the same split and metric definitions.
- Include efficiency (training / inference cost, model size) when relevant.

Recommended result table

Model	Validation score	Test score	Training time	Model size
Simple baseline	–	–	–	–
Conventional ML 1	–	–	–	–
Proposed method	–	–	–	–

Step 11: Error Analysis, Explainability and Robustness

Analyze failures

- Show representative correct and incorrect cases.
- Examine error by range, class and subgroup.
- Inspect residual / confusion patterns.
- Investigate unstable predictions.

Build confidence

- Feature importance, permutation importance or SHAP when justified.
- Ablations: remove features or components.
- Sensitivity to noise, shift and random seed.
- Explain limitations of interpretation.

Do not overclaim

Correlation or an attribution score alone is not proof of causation.

Step 12: Reproducibility, Ethics and Documentation

- Use a clear project folder: `data/`, `src/`, `notebooks/`, `results/`, `reports/`.
- Include a `README` with dataset access, commands and expected outputs.
- Save split indices, configurations, seeds and model artifacts where permitted.
- Provide `requirements.txt` or an equivalent environment specification.
- Discuss licensing, privacy, fairness, potential misuse and limitations.
- Verify that a fresh run reproduces tables and plots reasonably closely.

Reproducibility test

Mentor can regenerate the key results using your instructions without asking you for hidden steps?

Suggested Project Folder Structure

Example organization

```
project/
  README.md
  requirements.txt
  configs/
  data/ (or download instructions)
  notebooks/ (EDA only)
  src/ (processing, train, evaluate)
  results/ (tables and plots)
  models/ (when allowed)
  report/ and slides/
```

Quality expectations

- No hidden manual preprocessing.
- Modular, readable, commented code.
- Paths and settings configurable.
- Clear training and evaluation commands.
- No secrets or restricted data committed.

Common Mistakes to Avoid

1. Training, feature selection or preprocessing using test-set information.
2. Reporting only accuracy on an imbalanced dataset.
3. Comparing methods using different splits or incompatible metrics.
4. Claiming novelty or improvements without adequate baselines.
5. Reporting a single favorable run without uncertainty or failure analysis.
6. Presenting plots without labels, units, captions or interpretation.
7. Submitting notebooks or slides without reproducible code and documentation.

What to Include in the Final Project Report

1. Title, abstract and contributions.
2. Introduction and problem statement.
3. Related work and gap.
4. Dataset and quality audit.
5. Split protocol and methodology.
6. Experimental setup and baselines.
7. Results with tables / plots.
8. Error analysis and ablations.
9. Limitations, ethics and conclusion.
10. References and reproducibility notes.

Writing standard

Clearly separate what was planned, what was implemented, what the evidence shows and what remains uncertain.

Suggested Final Presentation: 10–15 Slides

Slide range	Content
1–2	Title, motivation, precise problem and objectives
3–4	Literature review, identified gap, dataset overview
5–6	Data audit, splits, preprocessing and leakage controls
7–8	Baselines, proposed workflow / algorithms, experiment protocol
9–11	Main results, comparison, plots and error analysis
12–13	Ablation / robustness, limitations and implications
14–15	Conclusion, future work and references / Q&A

Presentation rule

Explain your own work and diagrams clearly; every claim should be backed by an experiment or citation.

Project Completion Checklist

- ☐ Problem and metrics defined.
- ☐ Literature and gap documented.
- ☐ Dataset source and license recorded.
- ☐ Data audit and EDA completed.
- ☐ Leak-free split and pipeline verified.
- ☐ Simple baseline established.
- ☐ Models tuned using validation only.
- ☐ Held-out evaluation completed.
- ☐ Results and failures interpreted.
- ☐ Seeds, versions and commands recorded.
- ☐ Code, report and slides organized.
- ☐ Limitations and references included.

Final test

Correct protocol + fair comparison + reproducible evidence + clear communication = strong ML project.

How to Decide the Project Title

Guidelines

1. State the **task** and the **application/domain**.
2. Include the key approach only when it is central.
3. Be specific about the **prediction target**.
4. Prefer clear, concise wording (about 10–16 words).
5. Avoid vague claims such as "novel," "best," or "100% accurate."

Simple title formula

[Task] + [Approach] + [Application / Target]

Example: refinement

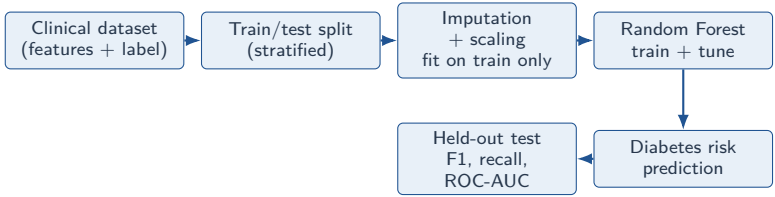
Too broad: Machine Learning in Healthcare

Better: Diabetes Risk Prediction Using Random Forest on Clinical Data

Title should accurately reflect the implemented work, not promise unverified performance.

How to Draw the ML Model Architecture Diagram

Guidelines: Use labeled blocks in data-flow order; show inputs, preprocessing, learning model, and output. Distinguish training from inference and indicate where fitting occurs.



Sample caption

Architecture of a diabetes-risk classification pipeline. Training data are used for preprocessing, model fitting and tuning; the held-out test set is used only for final evaluation.

How to Write the ML Project Algorithm

Guidelines

1. Give the algorithm a descriptive name.
2. Declare **Input** and **Output**.
3. Number steps in execution order.
4. Specify data split, preprocessing, training, tuning and evaluation.
5. Show loops/selection rules when relevant; avoid code-specific syntax.

Sample Algorithm: Diabetes Risk Prediction

Input: Labeled clinical dataset D, hyperparameter grid H
Output: Fitted classifier f^*, held-out metrics

1. Split D into stratified training set D_{tr} and test set D_{te}.
2. On D_{tr}, define a pipeline: imputation $\rightarrow$ scaling $\rightarrow$ Random Forest.
3. For each $h \in H$, estimate cross-validated F1 using folds of D_{tr}; fit preprocessing inside each fold.
4. Set $h^* = \arg\max_{h \in H} \mathrm{CVF1}(h)$.
5. Fit the complete pipeline f^* with h^* on all D_{tr}.
6. Predict labels/scores for D_{te}; report F1, recall and ROC-AUC.

Rule: Pseudocode must match the actual implementation and must not tune on test data.

Questions and Discussion

IMPACT Lab

Department of Computer Science and Engineering
Indian Institute of Technology Patna